



Inspiring Excellence

Department of Computer Science and Engineering

Course: CSE447: Cryptography and Cryptanalysis

Semester: Spring 2026

Project Report

Title: CryptoVax: Secure National Vaccine Management

Submitted To: FFU, FDZ

Group No: 10

Section: 02

Submission Date: 08.05.2026

Group Members

No.	Full Name	Student ID
1	Sayem Afridi	21301044
2	Masrura Nahian Nodi	22201421

Table of Contents

1. Introduction and System Overview	3
2. Login and Registration Module	5
3. User Data Encryption and Decryption	8
4. Password Hashing and Salting	10
5. Two-Factor Authentication (2FA)	11
6. Key Management Module	12
7. Post and Profile Management	13
8. Data Storage Security	14
9. Message Authentication Code (MAC)	15
10. Role-Based Access Control (RBAC)	16
11. Secure Session Management	17
12. GitHub Repository and Project Structure	18
13. Conclusion	19

1. Introduction and System Overview

This report documents the design, implementation, and security analysis of the CSE447 Lab Project. The system is a secure web application integrating multiple cryptographic protocols as required by the course specification. All encryption algorithms have been implemented from scratch without relying on built-in framework encryption functions.

1.1 Project Overview

This project is a secure vaccine management system and vaccine diary platform. It allows users to register, log in, and manage their sensitive personal data such as name, NID, birthdate, phone number, and password. Also, it allows users to post secured diary entries about their vaccination experiences. The main objective of this project is to ensure "zero-knowledge" database storage, which means even if the database gets fully compromised, no plaintext data can be extracted by any attacker.

1.2 Technology Stack

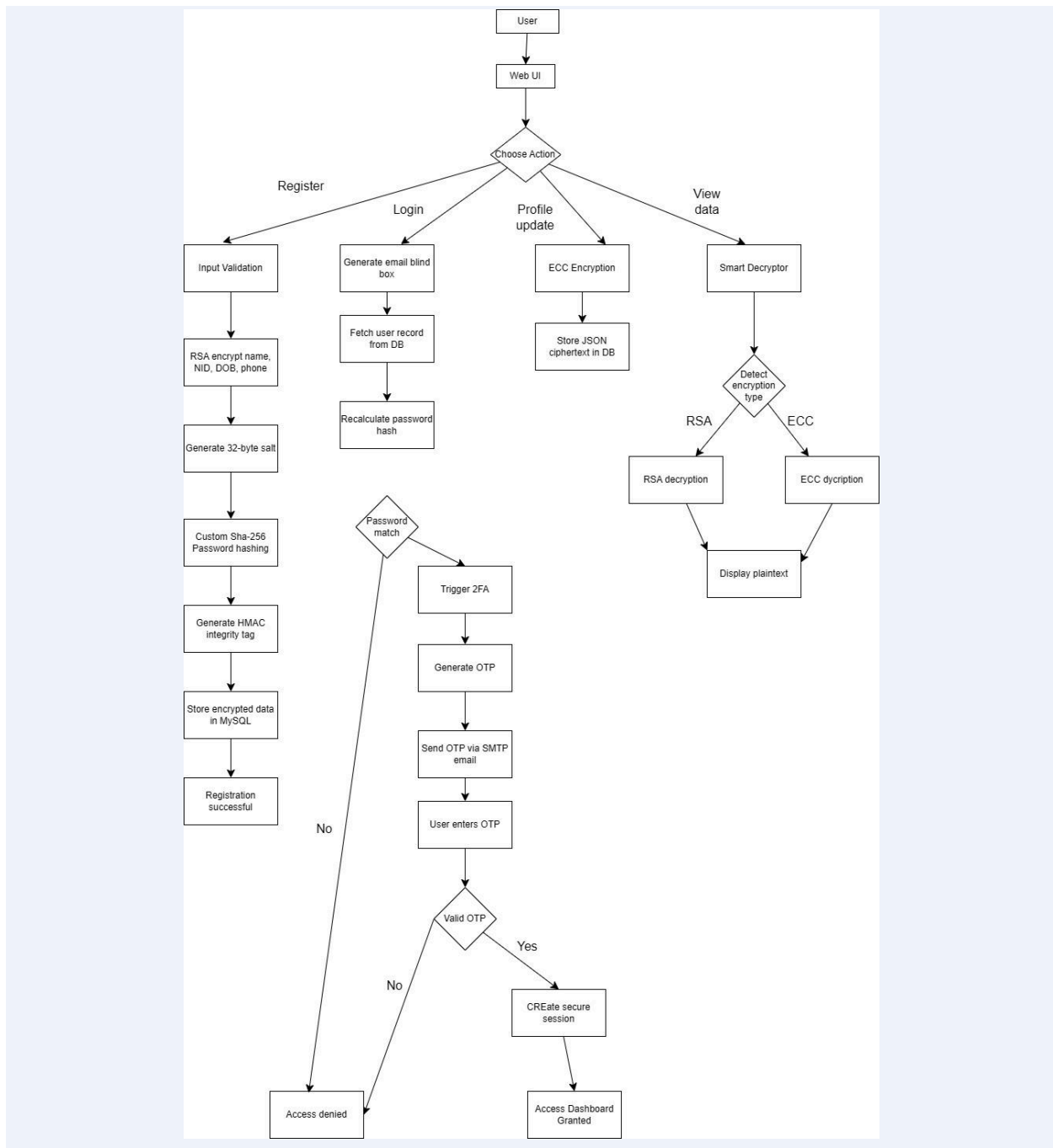
Backend: PHP 8.x

Framework: Laravel (Used for MVC architecture and routing, not for cryptography)

Database: MYSQL

Math extension: GMP—GNU Multiple Precision (For raw cryptography calculations from scratch)

1.3 System Architecture Diagram

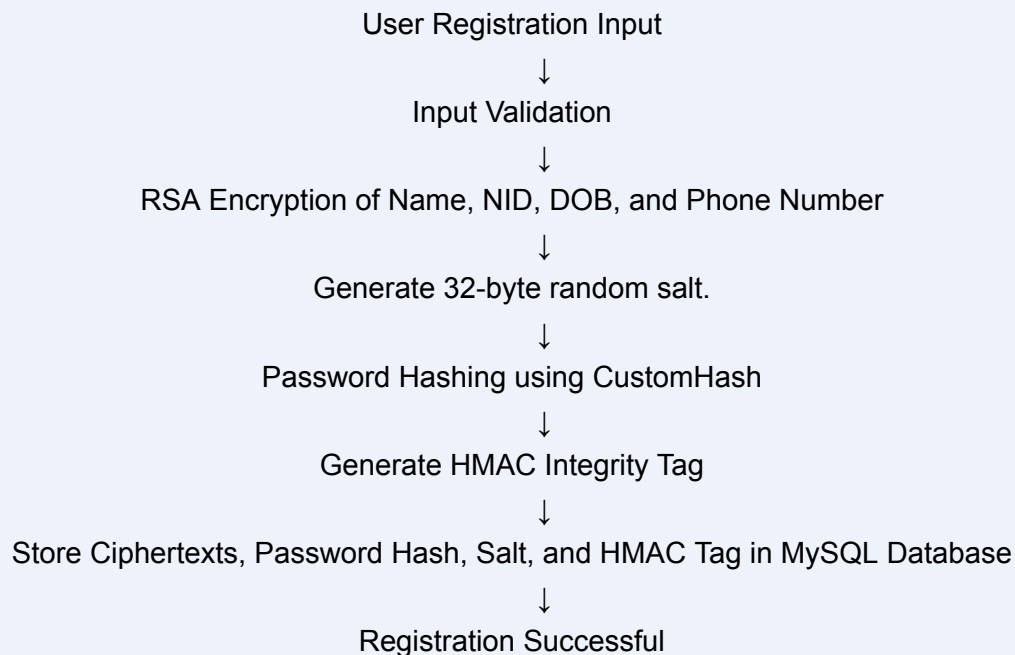


2. Login and Registration Module

The system provides secure registration and login flows. New users supply credentials, which are validated, encrypted, and persisted. During login, stored encrypted data is retrieved and decrypted for verification.

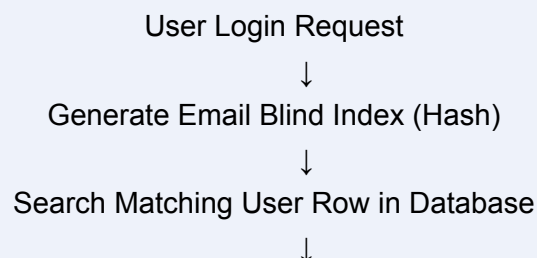
2.1 Registration Flow

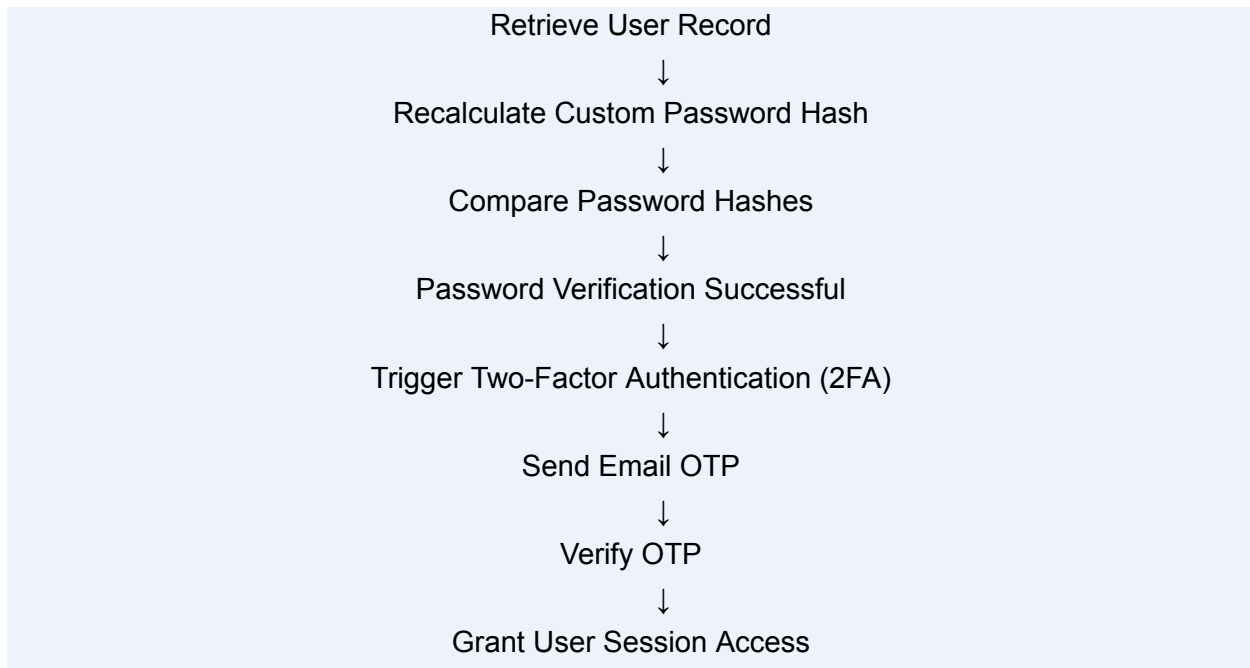
When a user registers, their plaintext data gets intercepted. The RSACrypto class encrypts the user's name, NID, date of birth, and phone number. The CustomHash class generates a 32-byte salt and hashes the password. HMACService generates a data integrity tag. Only the ciphertexts and tags are stored in the database.



2.2 Login Flow

An email hash (blind index) is used to securely query the database without decrypting every row because the email is used for routing. Once the user's row is found, the custom password hash is recalculated and compared. If it is successful, the system triggers the two-factor authentication (2FA) and sends an email OTP before granting a session.





2.3 Implementation Details

Requirement	Implementation Details
Login Module	• Uses a blind index (<i>email_hash</i>) to locate the user. • Verifies custom salted SHA-256 hash. • Triggers 2FA.
Registration Module	• Validates input format • Intercepts sensitive data and routes to the RSACrypto service before DB query is executed
Data Encrypted Before Storage	• Encrypt name, NID, DOB, and phone number using RSA during registration. • Discard plaintext values.
Data Decrypted on Retrieval	• Implement a smart decryptor accessor in the user model to decrypt the ciphertext on the fly when the application requests it.

3. User Data Encryption and Decryption

All sensitive user information (e.g., username, email, contact info) is encrypted before storage using asymmetric encryption algorithms implemented from scratch and decrypted upon retrieval.

3.1 Fields Encrypted

Field	Algorithm (registration)	Algorithm (update)
name_encrypted	RSA	ECC
nid_encrypted	RSA	ECC
dob_encrypted	RSA	ECC
phone_encrypted	RSA	ECC

3.2 Encryption Algorithm - RSA Implementation

RSA was implemented from scratch using PHP's GMP extension.

Here, two large prime numbers are generated: **p** and **q**.

The modulus is **$n=p \times q$** .

Euler's totient function, **$\phi(n)=(p-1)(q-1)$**

A public exponent **e** and private exponent **d** are generated to form the RSA key pair.

Encryption: Plaintext strings are converted into numeric representations. Ciphertext is produced by using the RSA encryption formula. **$c=m^e \pmod n$** . Here, **m** = plaintext message, **n** = modulus.

Decryption: The original message is recovered using **$m=c^d \pmod n$** . The resulting numeric message is decoded back to ASCII characters to reconstruct the original plaintext.

3.3 Encryption Algorithm - ECC Implementation

ECC (Elliptic Curve Cryptography) is implemented to handle profile updates.

ECC operates over an elliptic curve defined by the equation: **$y^2=x^3+ax+b \pmod p$**

Here, curve parameters are **a** and **b**.

Large number defining the finite field = **p**

The implementation includes custom point addition and scalar multiplication.

Encryption: It maps data to a point on the curve, generates a random ephemeral key, and outputs cipher components stored as a JSON object in the database.

3.4 How Both Algorithms Are Used Differently

To fulfill the requirement of using two asymmetric encryption algorithms, the system implements an algorithm rotation mechanism.

Registration phase: Use RSA (Initial data capture)

Profile update: Use ECC ("Smart Decryptor" dynamically detects if the ciphertext is an RSA base64 string or an ECC JSON object and applies the correct math engine).

4. Password Hashing and Salting

Passwords are never stored in plaintext. A cryptographic hash function combined with a random salt is applied before storage to prevent dictionary and rainbow-table attacks.

4.1 Hashing Algorithm Used

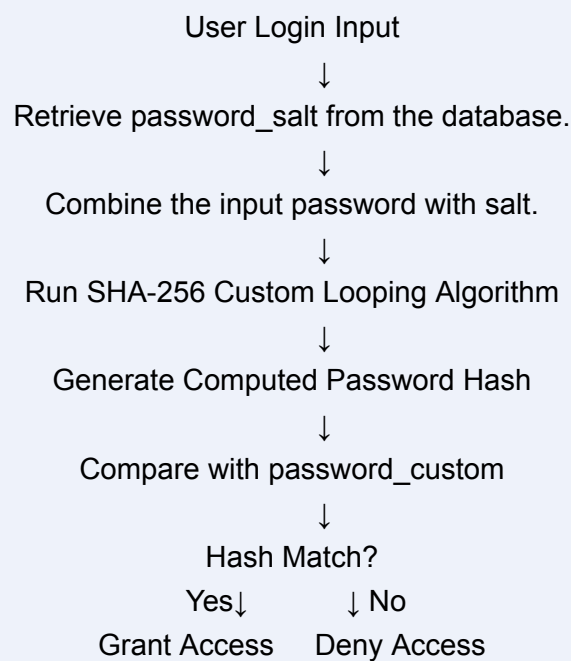
A custom hashing loop using SHA-256 was implemented instead of Laravel's default bcrypt. Multiple iterations are used to increase the computational cost of brute-force attacks.

4.2 Salt Generation

A secure 32-byte random salt is generated via `random_bytes()`. It is stored in a dedicated `password_salt` column alongside the custom hash.

4.3 Verification Process

During login, the system retrieves the `password_salt` from the database, matches it to the user's input password, and runs it through the identical SHA-256 custom looping algorithm. Access is granted only if the resulting hash matches `password_custom`.

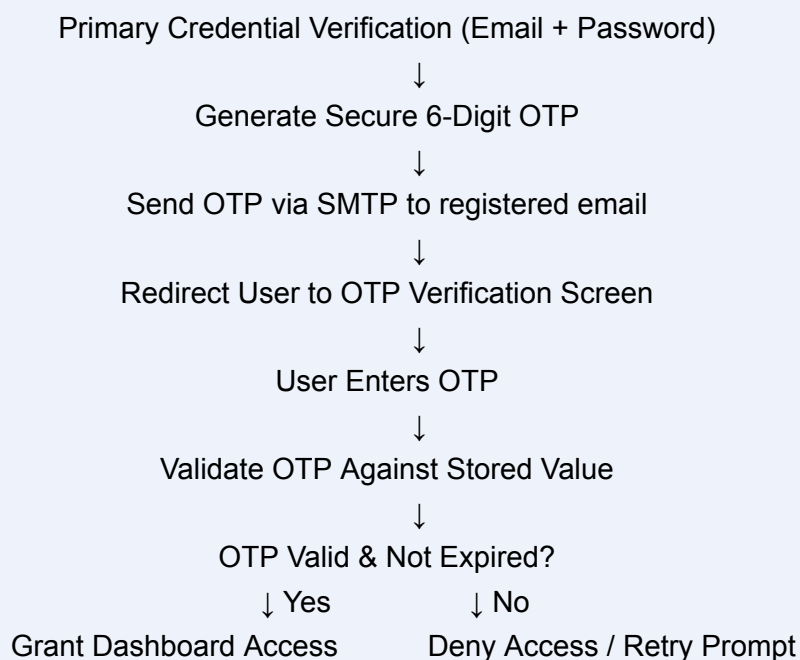


5. Two-Factor Authentication (2FA)

The system enforces two-step verification: the user must pass both primary credential validation and a second authentication factor before a session is granted.

5.1 2FA Method

After verifying the primary credentials (email/password), the system generates a secure 6-digit One-Time Password (OTP). This OTP is sent to the user's registered email via SMTP. The user is redirected to a verification screen and cannot access the dashboard until the matching code is provided within the expiration window.



5.2 Code Snippet

```
public function isOtpValid(string $code): bool
{
    return $this->otp_code === $code
        && $this->otp_expires_at
        && now()->lessThanOrEqualTo($this->otp_expires_at);
}
```

6. Key Management Module

A dedicated key management module handles the full lifecycle of cryptographic keys: secure storage and rotation.

6.1 Key Storage Security

The KeyManager class automatically generates the RSA and ECC key pairs upon system initialization. The public and private keys are stored in a dedicated `crypto_keys` table. The application dynamically fetches the active keys whenever an encryption or decryption operation is required.

6.2 Key Rotation Policy

Keys can be marked as inactive in the DB while a new row is created for fresh encryptions while still allowing old data to be decrypted using the archived keys.

7. Post and Profile Management

Users can create, view, and edit posts, as well as view and update their profiles. All post and profile data is automatically encrypted before storage and decrypted on retrieval.

7.1 Post Module

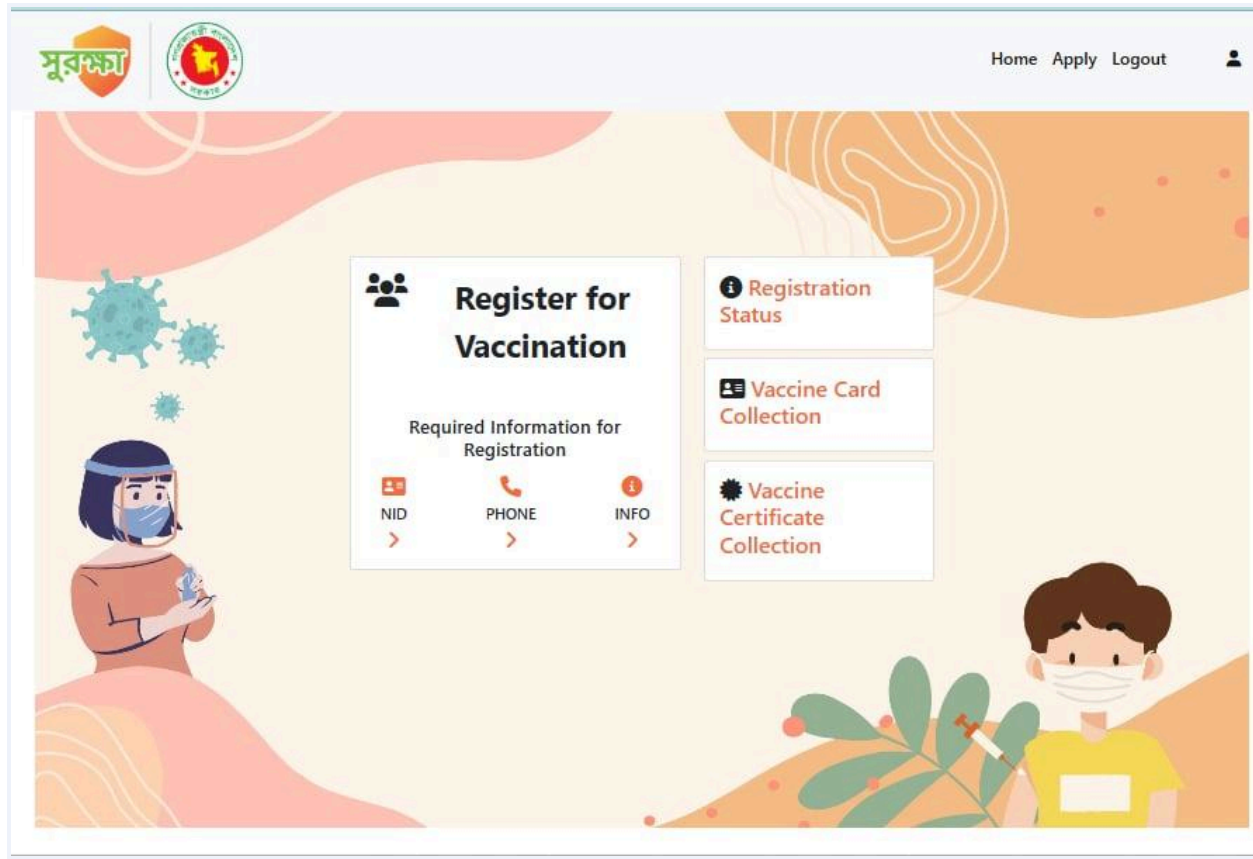
The "Vaccine Diary" allows users to create secure posts.

Field	Algorithm (initial creation)	Algorithm (editing posts)
title_encrypted	RSA	ECC
content_encrypted	RSA	ECC

7.2 Profile Module

The profile page displays the decrypted name, NID, phone, and DOB using the model's virtual accessors. When the user updates their profile, the Controller validates the new inputs, runs them through the ECCrypto class, and stores the newly generated JSON ciphertext.

7.3 Screenshots



Profile

Update Profile Information

Profile Information

Update your account's profile information, NID, and contact details. All sensitive data is encrypted using RSA/ECC.

Full Name

Email Address

National ID (NID)

Phone Number

Date of Birth

**SAVE CHANGES**

Update Password

Ensure your account is using a long, random password to stay secure.

Current Password

New Password

Confirm Password

SAVE

Delete Account

Once your account is deleted, all of its resources and data will be permanently deleted. Before deleting your account, please download any data or information that you wish to retain.

DELETE ACCOUNT

Vaccine Diary

Write a Secure Note

SAVE ENCRYPTED NOTE

Your Diary Entries

today

healthy.

[Edit \(Rotates to ECC\)](#)

8. Data Storage Security

All critical data - user information, posts, and cryptographic keys - is stored in encrypted form to prevent plaintext access even in the event of a database compromise.

8.1 Evidence of Encrypted Storage

Server: 127.0.0.1 » Database: vaccine_management » Table: users		
Browse	Structure	SQL
Search	Insert	Export
Import	Privileges	Operations
Triggers		
password_salt	password_custom	mac_tag
NULL	NULL	NULL
fb04cfe5de64a2e79c1660a808b6bfa704b608f5a869e1d9b...	24e39dc6700e76780ff86d59b9a0d240cd9117b89cf3975b2b...	5277f820fc55938a25b9dc0b691d8a58e4219711e263600e...
1f6034a27e01e01acbf9bdf9c753567e12d17e24e81d25a01...	04f5ff06da52c993d65562fff4e55115df3b9eb94c88c062fa...	d71de272aac25386a5f6045217376f7d084850b0f77c6249...
55a4fcacd361559f6506399762c10c8befdb71ba843b28106b...	4234dad2f3f32f899ed1ddfa56b5fc3eb46dd4c6c9b82a901...	8d5890bc0ead89ba11acbe12a53f9a062f95211cc3b09c83f...
bd0b001bf0af2ffdd2dcacbe5e8c72be2d910ba7c5344c1a1c...	6ee20a8f8f86e05366f2d300a8c37d40f7c3b9a68d71ab1bd...	ba12f139350781f022f0e3cc7ec0c1cef2b0e5dedc0a046e0...
0bf344a5f1ce4b45fb78f33d8e1a63649d961dd09fafc71002...	236f1f8a94151cf352b13b1c0006a1e37e034b7fcb29ed98b0...	2efd4c2600f16f0a04c2190756deac3183897aaa61a08d57f...
5accf24ac9fd02f0f4ddb4689da58adf4c190ac725a9efd8a...	6e97304385dd2b025ad85970b94f329f9a075b265fb0e4d12...	4657bc711ace6d377e500f8301c91e4fad07d5cb280fae995...
515cee33af5719f659f737d4cc6410a376910c3ab3d673dcad...	42d21d408f65fc33133ffda8ff9f6a764ddccc84f3a087f07...	188453a723e450541559705fd43520bba1739db974bcebcf...

Server: 127.0.0.1 » Database: vaccine_management » Table: users		
Browse	Structure	SQL
Search	Insert	Export
Import	Privileges	Operations
Triggers		
dob_encrypted	phone_encrypted	email_hash
NULL	NULL	NULL
{"ciphertext":"","ephemeral_pub_x":"1854667819","e...	{"ciphertext":"","ephemeral_pub_x":"1071022883","e...	4b7cb98c3529f837fb2afba9d894bb798e...
{"ciphertext":"","ephemeral_pub_x":"620274028","ep...	{"ciphertext":"","ephemeral_pub_x":"468543529","ep...	5e628bc437ec9a9a51e6d5ab4cb1fdb59...
{"ciphertext":"","ephemeral_pub_x":"2061556755","e...	{"ciphertext":"","ephemeral_pub_x":"1876190225","e...	bc109b6bcdce62eb7975bed1205513861...
ODgxNzU5NDIxMzY1OTIzMzY5NDQ1ODc3NzQ5MzM0Mzk1MDY2fD...	MzU4OTY2MjkwOTY1MjIzOTk0OTk0MTAwOTY2NDg4Nz...	29868e71630dcc712dae9c1f2327a8edd...
MTMwNTMwNTI5MzQ3MzY5NDQ1ODc3NzQ5MzM0Mzk1MDY2fD...	MTA1ODY2MjAwNTY1NzE1NzE2MzQ3MTM2NzY5MDYxMDkxMTg0NTg5Mj...	261cb21953c314677061f560b9e489105...
{"ciphertext":"pUmlaeun1OFI6w=","ephemeral_pub_x"...	{"ciphertext":"Z881eGIfUnU=","ephemeral_pub_x":"20...	e756fe8dcc9efb8995c3fb1fc107515a084...
{"ciphertext":"IM8qnXwXIIHJ0A=","ephemeral_pub_x"...	{"ciphertext":"PVuvzHlcPw=","ephemeral_pub_x":"14...	95436c19dd8ba0bd5bfd03c6dda72df7d...

Server: 127.0.0.1 » Database: vaccine_management » Table: users

name_encrypted	email_encrypted	nid_encrypted	dob_encrypted
NULL	NULL	NULL	NULL
{ "ciphertext": "yxVc+A==", "ephemeral_pub_x": "87621..." }	NULL	MTA1ODY2MjAwNzE1NzE2MzQ3MTM2NzM2MDYxMDkxMTg0NTg5Mj...	{ "ciphertext": "", "ephemeral_pub_x": "87621..." }
{ "ciphertext": "LFWWe8qnjC+Gy0=", "ephemeral_pub_x": "..." }	NULL	NjA4MjYxODU2MjA5NzEwOTIwNDI1NTg0NzcyMTgzMTQ5MzE0MD...	{ "ciphertext": "", "ephemeral_pub_x": "..." }
{ "ciphertext": "AWnZdo4=", "ephemeral_pub_x": "147042..." }	NULL	MTA1ODY2MjAwNzE1NzE2MzQ3MTM2NzM2MDYxMDkxMTg0NTg5Mj...	{ "ciphertext": "", "ephemeral_pub_x": "147042..." }
Mjk3Njg2MTkyNzk1NjY2MjU0NTEwMDU0MTA1MjA5NTc1OTkzND...	NULL	MTA1ODY2MjAwNzE1NzE2MzQ3MTM2NzM2MDYxMDkxMTg0NTg5Mj...	ODgxNzU5NDIxMzY0MTA1MjA5NTc1OTkzND...
ODAzNjg2MTkyNzk1NjY2MjU0NTEwMDU0MTA1MjA5NTc1OTkzND...	NULL	MTA1ODY2MjAwNzE1NzE2MzQ3MTM2NzM2MDYxMDkxMTg0NTg5Mj...	MTMwNTMwNzI1MzY0MTA1MjA5NTc1OTkzND...
{ "ciphertext": "URjjP+gP", "ephemeral_pub_x": "160824..." }	NULL	{ "ciphertext": "oXOwHLryfnk=", "ephemeral_pub_x": "16..." }	{ "ciphertext": "pUmla...", "ephemeral_pub_x": "160824..." }
{ "ciphertext": "Ali2AK9n", "ephemeral_pub_x": "133536..." }	NULL	{ "ciphertext": "HriT0i0=", "ephemeral_pub_x": "395849..." }	{ "ciphertext": "lM8qn...", "ephemeral_pub_x": "133536..." }

Console

9. Message Authentication Code (MAC)

Message Authentication Codes (MACs) are used to verify the integrity of stored data and detect any unauthorized modifications. The system implements CBC-MAC or HMAC as required.

9.1 MAC Algorithm Used

An HMAC implementation utilizing SHA-256 was created from scratch. It acts as a digital seal over the user's data and posts.

9.2 Integrity Verification Flow

Upon data creation or update, a `mac_tag` is generated based on the combination of the input fields. When the user's profile or post is loaded, the system recalculates the MAC. If an attacker modifies the ciphertext directly in the database, the MACs will fail to match, triggering a security alert to the user.

10. Role-Based Access Control (RBAC)

Role-based access control defines distinct privilege levels for administrators and regular users, ensuring that sensitive operations are restricted appropriately.

10.1 Roles Defined

Regular User = 0: Can only view/edit their own profile and posts.

Admin = 1: Has privileges and alternate routing

10.2 Permission Matrix

Operation / Resource	Admin	Regular User
View own profile	✓	✓
Edit own profile	✓	✓
Create / Edit posts	✓	✓
View admin dashboard	✓	✗

11. Secure Session Management

Authentication tokens and session identifiers are managed securely to prevent session hijacking, fixation, and replay attacks.

11.1 Token Signing / Verification

A custom SecureSession middleware was implemented. Upon login, the system creates a composite hash of the user's IP address and browser user agent. On every subsequent HTTP request, this hash is recalculated. If an attacker steals the session cookie and attempts to use it from a different device, the hash fails, and the session is immediately invalidated.

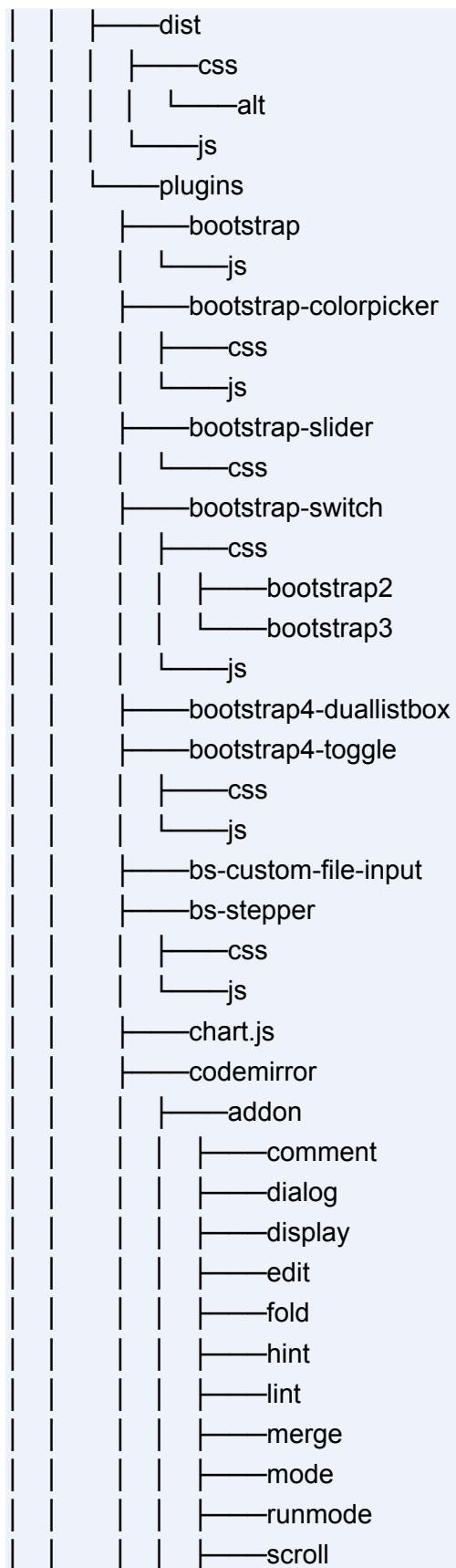
12. GitHub Repository and Project Structure

Field	Details
GitHub Repository URL	https://github.com/SayemAfridi07/CryptoVax--Secure-National-Vaccine-Management-System

12.1 Repository Structure

C:.

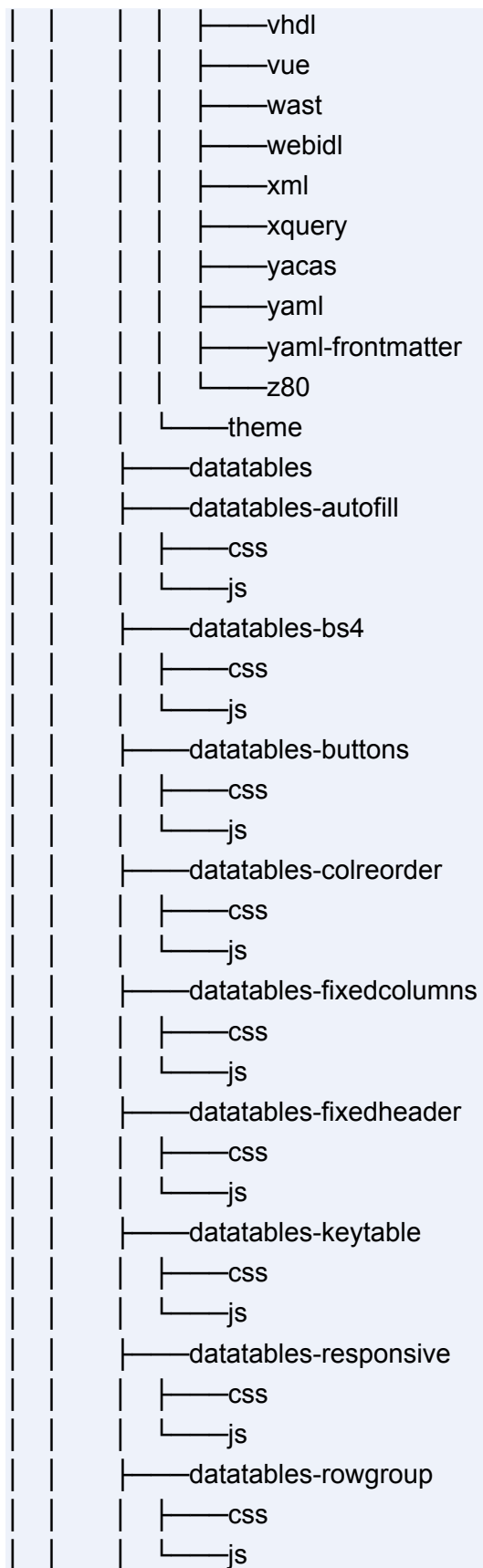
```
|— app
|   |— Console
|   |   |— Commands
|   |— Http
|   |   |— Controllers
|   |       |— Admin
|   |       |— Auth
|   |       |— Front
|   |       |— Operator
|   |   |— Middleware
|   |   |— Requests
|   |       |— Auth
|   |— Mail
|   |— Models
|   |— Providers
|   |— Services
|   |   |— Crypto
|   |— View
|   |   |— Components
|— bootstrap
|   |— cache
|— config
|— database
|   |— factories
|   |— migrations
|   |— seeders
|— public
|— assets
```



					search
					selection
					tern
					wrap
					keymap
					mode
					apl
					asciarmor
					asn.1
					asterisk
					brainfuck
					clike
					clojure
					cmake
					cobol
					coffeescript
					commonlisp
					crystal
					css
					cypher
					d
					dart
					diff
					django
					dockerfile
					dtd
					dylan
					ebnf
					ecl
					eiffel
					elm
					erlang
					factor
					fcl
					forth
					fortran
					gas
					gfm
					gherkin

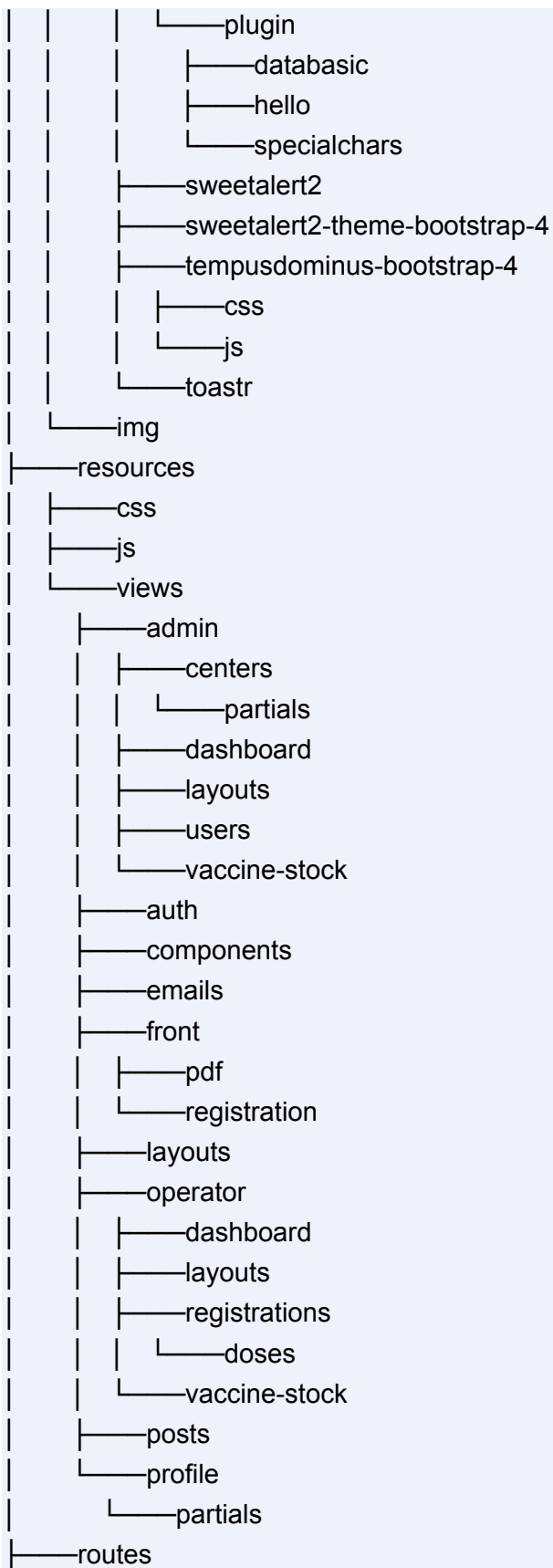
					—go
					—groovy
					—haml
					—handlebars
					—haskell
					—haskell-literate
					—haxe
					—htmlembedded
					—htmlmixed
					—http
					—idl
					—javascript
					—jinja2
					—jsx
					—julia
					—livescript
					—lua
					—markdown
					—mathematica
					—mbox
					—mirc
					—mllike
					—modelica
					—mscgen
					—mumps
					—nginx
					—nsis
					—ntriples
					—octave
					—oz
					—pascal
					—pegjs
					—perl
					—php
					—pig
					—powershell
					—properties
					—protobuf
					—pug

					—puppet
					—python
					—q
					—r
					—rpm
					—rst
					—ruby
					—rust
					—sas
					—sass
					—scheme
					—shell
					—sieve
					—slim
					—smalltalk
					—smarty
					—solr
					—soy
					—sparql
					—spreadsheet
					—sql
					—stex
					—stylus
					—swift
					—tcl
					—textile
					—tiddlywiki
					—tiki
					—toml
					—tornado
					—troff
					—ttcn
					—ttcn-cfg
					—turtle
					—twig
					—vb
					—vbscript
					—velocity
					—verilog



- └─ datatables-rowreorder
 - └─ css
 - └─ js
- └─ datatables-scroller
 - └─ css
 - └─ js
- └─ datatables-select
 - └─ css
 - └─ js
- └─ daterangepicker
- └─ dropzone
 - └─ min
- └─ ekko-lightbox
- └─ fastclick
- └─ filterizr
- └─ flag-icon-css
 - └─ css
 - └─ flags
 - └─ 1x1
 - └─ 4x3
- └─ flot
 - └─ plugins
- └─ fontawesome-free
 - └─ css
 - └─ webfonts
- └─ fullcalendar
 - └─ locales
- └─ icode-bootstrap
- └─ inputmask
- └─ ion-rangeslider
 - └─ css
 - └─ js
- └─ jquery
- └─ jquery-knob
- └─ jquery-mapael
 - └─ maps
- └─ jquery-mousewheel
- └─ jquery-ui
 - └─ images

- └─jquery-validation
 - └─localization
- └─jqvmap
 - └─maps
 - └─continents
- └─jsgrid
 - └─demos
 - └─i18n
- └─jszip
- └─moment
 - └─locale
- └─overlayScrollbars
 - └─css
 - └─js
- └─pace-progress
 - └─themes
 - └─black
 - └─blue
 - └─green
 - └─orange
 - └─pink
 - └─purple
 - └─red
 - └─silver
 - └─white
 - └─yellow
- └─popper
 - └─esm
 - └─umd
- └─raphael
- └─select2
 - └─css
 - └─js
 - └─i18n
- └─select2-bootstrap4-theme
- └─sparklines
- └─summernote
 - └─font
 - └─lang



```
|—storage
| |—app
| | |—private
| | |—public
| |—framework
| | |—cache
| | |—data
| | |—sessions
| | |—testing
| | |—views
| |—logs
|—tests
| |—Feature
| | |—Auth
| |—Unit
```

[Paste or describe the top-level folder/file structure of your repository (e.g., output of `tree` command or a directory listing).]

12.2 README Overview

This web application is intended to make national vaccination operations smooth, secure, and automated. It provides a seamless portal for both administrators managing the vaccine supply chain and citizens booking their vaccination appointments. Beyond its core functionality, this system has been fortified with custom, from-scratch cryptographic protocols to ensure maximum data privacy. The database operates on a "zero-plaintext" policy, meaning no sensitive citizen data is ever stored in a readable format.

This project features heavy security modifications, bypassing standard framework wrappers to implement raw cryptography:

Zero-Plaintext Database: All sensitive columns (name, NID, DOB, and phone) have been physically removed and replaced with encrypted ciphertexts.

Algorithm Rotation (RSA & ECC): User data is encrypted using RSA upon registration. When a profile is updated, the system re-encrypts the data using Elliptic Curve Cryptography (ECC) to introduce fresh cipher randomness.

Data Integrity (HMAC): Every sensitive database row is sealed with a custom Hash-based message authentication code (HMAC). Tampering with the ciphertext in the database triggers an automated integrity warning.

Custom Password Hashing: Standard bcrypt is replaced with a custom-built, heavily looped SHA-256 algorithm utilizing a random 32-byte salt stored independently.

Two-Factor Authentication (2FA): Access requires validating a 6-digit OTP sent securely via SMTP to the user's verified email address.

Session Hijacking Prevention: Sessions are dynamically bound to the user's IP and browser hardware fingerprint.

Core Features

Admin Panel (For Higher Authority)

Center Management: The admin manages vaccine centers.

Supply Chain: Admin supplies vaccines (vials) to the centers.

Staffing: Admin assigns and authorizes healthcare practitioners.

Vaccine Candidates (Citizen Portal)

Registration: Register securely on the platform.

Applications: Apply for specific vaccines based on availability.

Scheduling: Set and manage appointments.

Documentation: Download digital vaccine cards and certificates.

Vaccine Diary: Create end-to-end encrypted medical diary entries/posts.

System Dependencies

To run this project, ensure that the environment meets the following requirements:

PHP: 8.2^

Required PHP Extensions: gd, mbstring, zip, gmp (GNU Multiple Precision for Cryptography)

MySQL: 8.0^

Node.js: 18.20.3^

NPM: 10.7.0^

Note: Other missing dependencies will be prompted when the composer install command is run.

Installation Steps

Run the following commands in your project directory to get the application up and running:

Environment Setup:

```
cp .env.example .env
```

Make sure to open .env and add your database credentials and SMTP email credentials (required for 2FA).

Install PHP Dependencies:

Bash composer install Generate App Key:

Bash php artisan key:generate Run Migrations & Seeders:

Bash php artisan migrate php artisan db:seed, install & Compile Frontend Assets:

Bash npm install npm run dev

Use npm run build for production

Testing Accounts & Credentials To explore the system, you can use the following seeded accounts:

Admin Account:

Email: admin@test.com

Password: password

Operator Account:

Email: operator@test.com

Password: password

Note: Applicant accounts can be registered right away via the registration page, and no manual verification is needed for basic services.

13. Conclusion

Developing the vaccine management system provided profound, hands-on experience with applied cryptography. By implementing RSA, ECC, and custom hashing from scratch without framework shortcuts, we gained a deep understanding of the mathematical foundations of cybersecurity. Overcoming the challenges of querying a zero-knowledge database and implementing seamless "algorithm rotation" successfully demonstrated that high-level security does not have to compromise user experience.